

Resumen Ejecutivo

Desarrollar un micro-SaaS "AI Study & CV Booster" con Java (Spring Boot) + React permite crear una **herramienta práctica** de apoyo académico y profesional basada en IA. Casos de uso típicos incluyen generación automática de resúmenes de texto, creación de tests o flashcards para estudio, y optimización de CV/resume basándose en inteligencia artificial. Empresas de edtech ya usan IA para "resumir contenido y generar flashcards" para alumnos ¹, así como "análisis y generación de CV e informes para reclutamiento" ². De hecho, OpenAI identifica la educación como un "campo fronterizo prometedor" donde la IA ofrece apoyo personalizado para cualquier estudiante ³. Por tanto, esta micro-SaaS puede ayudar a estudiantes y profesionales a acelerar su aprendizaje y mejorar su CV, siguiendo ejemplos de eficiencia comprobada (80% ganancia de eficiencia en procesos de CV) ⁴ ³.

1. Visión general del producto y casos de uso

- **Objetivo:** Crear un MVP que ofrezca a usuarios (estudiantes, recién graduados) funcionalidades como generación de resumen de texto, flashcards de estudio, preguntas de auto-evaluación, y optimización/formateo de CV.
- **Casos de uso principales:**
 - **Estudio Personalizado:** Ingresar apuntes o textos (PDF, URL, etc.) y recibir resúmenes estructurados, tarjetas de estudio (flashcards) o preguntas de prueba. OpenAI ha mostrado mejoras en el aprendizaje con "Study mode" basado en IA ³, validando este enfoque.
 - **Preparación de CV:** Ingresar datos personales y laborales, y generar un CV optimizado con lenguaje profesional. Incluso una variante puede analizar un CV existente y sugerir mejoras. Cognizant reporta herramientas que generaron resúmenes de CV y preguntas de entrevista, aumentando un 80% la eficiencia ⁴.
 - **Usuarios objetivo:** Estudiantes universitarios, profesionales junior, y reclutadores ligeros. El producto se usará ocasionalmente (prep. examen o generación de CV) pero con potencial de crecimiento si se automatiza.

2. Recursos oficiales y tutoriales recomendados

- **Java Spring Boot:** Documentación oficial "Getting Started" ⁵. Tutorial "Accessing Data with JPA" para modelar datos ⁶.
- **Seguridad (JWT):** Guía de Spring Security con JWT en español (JoseAlopez.dev) ⁷. También Baeldung sobre CORS/Preflight ⁸.
- **Spring Data JPA:** Documentación oficial de Spring Data JPA ⁶ (anotaciones como `@Entity`, `@Id`).
- **React (Vite):** Documentación React en español (es.react.dev) ⁹. Guía oficial "Crear aplicación con Vite" ¹⁰ (Vite acelera desarrollo). Tutoriales React+Vite.
- **Tailwind CSS:** Tutorial introducción en español ¹¹ ("Tailwind... te garantiza crear interfaces hermosas y potentes").
- **Axios:** Guía en español de DigitalOcean ¹² (Axios es un *cliente HTTP ligero basado en promesas, similar a Fetch*).
- **OpenAI API:** Documentación oficial (OpenAI API Reference) ¹³ con ejemplo de llamada `curl` usando modelo `gpt-3.5-turbo` con clave API en header.

- **HuggingFace Inference:** Documentación de la Inference API ¹⁴ ("HTTP requests con tu modelo, token HF").
- **PostgreSQL:** Sitio oficial ¹⁵ ("PostgreSQL es un sistema de base de datos relacional de código abierto, confiable y de alto rendimiento"). Tutorial de conexión con Spring Boot (p. ej. Medium o Baeldung).
- **Despliegue (Railway/Render/Vercel):**
- **Railway:** Guía oficial para Spring Boot ¹⁶ (despliegue sencillo con one-click o CLI).
- **Render:** Documentación similar (no citada aquí, pero existe guía en docs.render.com).
- **Vercel:** Guía oficial React en Vercel ¹⁷ ("una sola plataforma... single-command deployment, CDN global, SSL automático"). También puedes usar Netlify o Heroku.

3. Arquitectura detallada

El sistema es una aplicación web tipo SPA (Single Page App) con **frontend React** y **backend Spring Boot**, conectados por HTTP REST. El backend se comunica con la **API de OpenAI** (u otra, e.g. Hugging Face) para tareas de IA, y con una **base de datos PostgreSQL** para almacenar usuarios, sesiones de uso y datos de CV.

```

flowchart LR
    subgraph Cliente
        A[Usuario] --> B[React (Vite) Frontend]
    end
    subgraph Servidor
        B --> C[Spring Boot API (Java)]
        C --> D[(PostgreSQL DB)]
        C --> E[OpenAI API / HuggingFace API]
        D --> C
    end

```

- **Componentes:** React (Vite) en frontend, Spring Boot con **Spring MVC** y **Spring Security JWT** en backend. Base de datos Postgres para persistir usuarios, CVs, registros. La IA (OpenAI/ HuggingFace) se invoca desde el backend con llamada HTTP POST.
- **Flujo de datos:** El usuario interactúa con React; React envía solicitudes a endpoints REST del backend (por ejemplo, `/api/cv`, `/api/study`). El backend en `@RestController` recibe peticiones, valida JWT, guarda/retrieves datos (JPA `@Entities`) y llama al servicio IA con `RestTemplate` o `WebClient`. La respuesta (JSON) se retorna a React.
- **Despliegue:** El frontend puede alojarse en Vercel/Netlify, el backend en Railway/Render, ambos enlazados a un repositorio GitHub. Ejemplo de despliegue:

```

flowchart TB
    CodeRepo[GitHub Repo] --> Vercel[Frontend (Vercel Deployment)]
    CodeRepo --> Railway[Backend (Railway Deployment)]
    Railway --> DB[(PostgreSQL en Railway)]
    Railway --> OpenAI[(OpenAI API)]
    Railway --> HF[(HuggingFace API)]

```

4. Modelo de datos SQL

Entidades principales (ejemplos):

Tabla	users	Tipo	Descripción	Relaciones
	id	UUID	PK, identificador	
	email	String	Usuario, único	
	password_hash	String	Hash de contraseña	
	name	String	Nombre completo	
	created_at	Timestamp	Fecha de registro	

Tabla	cv_profiles	Tipo	Descripción	Relaciones
	id	UUID	PK	
	user_id	UUID	FK -> users.id	uno-a-muchos con <code>users</code>
	raw_text	Text	Contenido original del CV	
	generated_cv	Text	CV generado (output)	
	created_at	Timestamp	Creación	

Tabla	study_requests	Tipo	Descripción	Relaciones
	id	UUID	PK	
	user_id	UUID	FK -> users.id	uno-a-muchos con <code>users</code>
	input_text	Text	Contenido para resumir/ generar	
	output_summary	Text	Resumen generado	
	flashcards	JSON	Tarjetas/Quizz	
	created_at	Timestamp	Creación	

(Ejemplo de migraciones JPA):

```
CREATE TABLE users (  
  id UUID PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  name VARCHAR(255),  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```

CREATE TABLE cv_profiles (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL,
  raw_text TEXT,
  generated_cv TEXT,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id)
);

CREATE TABLE study_requests (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL,
  input_text TEXT,
  output_summary TEXT,
  flashcards JSON,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id)
);

```

5. Endpoints REST

Las rutas REST siguen convención `/api/...` con peticiones JSON. Ejemplos:

Ruta	Método	Descripción	Request Body	Response & Códigos
<code>POST /api/auth/signup</code>	POST	Registrar nuevo usuario	<code>{ "email", "password", "name" }</code>	201 Created o 400 error validaci
<code>POST /api/auth/login</code>	POST	Iniciar sesión (devuelve JWT)	<code>{ "email", "password" }</code>	200 OK + <code>{token}</code> o 401 si falla
<code>GET /api/me</code>	GET	Datos del usuario autenticado	(header Authorization)	200 OK + <code>{id,email,name}</code> , 40 no valido
<code>POST /api/cv/generate</code>	POST	Generar CV optimizado	<code>{ "rawText": "...", "options":{...} }</code>	200 + <code>{ "cv": "..."} </code> o errores 400/500
<code>POST /api/study/summary</code>	POST	Resumir texto / generar flashcards	<code>{ "text": "...", "mode":"flashcards" }</code>	200 + <code>{summary, flashcards, questions}</code> o 400
<code>GET /api/history</code>	GET	Obtener historial de solicitudes	(autenticado)	200 + <code>[{type,input,output,date}, .</code>

Validaciones & errores típicos: Uso de anotaciones Spring `@Valid`, manejo de excepciones con controladores globales. Ejemplo: 400 si falta campo, 401/403 en JWT inválido, 500 si falla servicio IA.

6. Estructura del proyecto Backend (Spring Boot)

Paquetes sugeridos:

- `com.app.auth` (controladores `AuthController`, servicios `AuthService`, entidades `User`, repositorio `UserRepository`).
- `com.app.cv` (controlador `CvController`, servicio `CvService`, entidad `CvProfile`, `CvRepository`).
- `com.app.study` (controlador `StudyController`, servicio `StudyService`, entidad `StudyRequest`, `StudyRepository`).
- `com.app.security` (configuración `SecurityConfig`, filtros JWT `JwtAuthenticationFilter`, utilidades de token).

Clase clave: `SecurityConfig` con `HttpSecurity` configura endpoints públicos (`/api/auth/**`) vs protegidos, y define filtro JWT extendiendo `OncePerRequestFilter` ⁷.

Ejemplo pseudocódigo (Controlador CV):

```
@RestController
@RequestMapping("/api/cv")
public class CvController {
    @Autowired CvService cvService;

    @PostMapping("/generate")
    public ResponseEntity<CvResponse> generateCv(@RequestBody @Valid
    CvRequest req) {
        String generated = cvService.generateCv(req.getRawText(),
        req.getOptions());
        return ResponseEntity.ok(new CvResponse(generated));
    }
}
```

Servicio CvService: llama a OpenAI, por ejemplo con `RestTemplate`:

```
public String generateCv(String rawText, Options options) {
    // Construir prompt
    String prompt = "Generate a professional CV from the following info: " +
    rawText;
    // Preparar petición
    HttpEntity<ChatRequest> request = new HttpEntity<>(new
    ChatRequest("gpt-5.4-mini", prompt));
    ChatResponse response = restTemplate.postForObject(openAiApiUrl,
    request, ChatResponse.class);
}
```

```

    return response.getChoices().get(0).getMessage().getContent();
}

```

Aquí `restTemplate` está configurado con interceptor que añade el header `Authorization: Bearer ${API_KEY}` ¹³.

7. Estructura del proyecto Frontend (React + Vite)

Carpeta típica:

```

/src
  /api          # funciones de llamada Axios a backend (p. ej. api.js)
  /components   # componentes UI (p. ej. LoginForm, CvForm, SummaryView,
FlashcardList)
  /pages        # páginas principales (Login, Dashboard, Profile)
  /hooks        # hooks personalizados (useAuth, useApi)
  /context      # contexto para auth/global state
  /App.js       # enrutamiento y layout general

```

- **Autenticación JWT:** Tras login, guardar JWT preferentemente en `HttpOnly` cookie (más seguro). Si no, almacenar en `localStorage` con precaución. Un `hook` `useAuth` puede manejar login/logout.
- **Llamadas API:** Se usa `axios` con baseUrl al backend. Ejemplo de un hook para obtener resumen:

```

import axios from 'axios';
export function useSummary() {
  const [result, setResult] = useState(null);
  const callSummary = async (text) => {
    try {
      const res = await axios.post('/api/study/summary', { text }, {
        headers: { Authorization: `Bearer ${token}` }
      });
      setResult(res.data);
    } catch (err) { console.error(err); }
  };
  return { result, callSummary };
}

```

- **Componentes clave:** Form de login/register, formulario de generación de CV (`<CvForm>`), área de estudio (`<StudyForm>` para texto y botones). Páginas: `<LoginPage>`, `<Dashboard>` con menús (Summary, CV, Historial).
- **Estilos:** Se usa Tailwind CSS para diseño rápido de UI.

8. Prompts optimizados (guiones para IA)

Se recomiendan **prompts claros y específicos** para cada tarea. Por ejemplo:

- **Curriculum (CV):**

"Act as a career advisor. From this input text `{rawText}`, generate a professional résumé/CV in Spanish with clear sections (Educación, Experiencia, Habilidades) and bullets describing achievements. Mantén un tono formal."

- **Resumen de texto:**

"Resume en español el siguiente texto enfatizando los puntos clave y conceptos importantes: `{texto}`."

- **Flashcards/Test:**

"Lee este texto `{texto}` y genera una lista de *preguntas de test* de opción múltiple (y sus respuestas correctas) enfocadas en los conceptos clave para repasar."

- **Explicaciones o consejos de estudio:**

"Genera una breve explicación o analogía para entender mejor este concepto: `{concepto}`."

- **Generación de tests:**

"Genera 5 preguntas con 4 opciones cada una sobre `{tema}`. Marca la respuesta correcta."

- **Trucos:** Usar prompts con instrucciones al sistema ("Actúa como profesor..."), y experimentar con parámetros (p. ej. `max_tokens` para controlar longitud).

9. Plan de desarrollo por semanas (30–60 min/día)

A continuación, un cronograma estimado de 4 semanas (día a día, ~30 min):

```
gantt
    title Plan de desarrollo (4 semanas)
    dateFormat YYYY-MM-DD
    section Semana 1
    Configuración inicial (Spring Boot, React, DB):      2026-05-04, 3d
    Modelado de datos y entidades (JPA):                  after Configuración
    inicial, 2d
    Endpoints usuarios (signup/login):                    2026-05-08, 2d
    Frontend login/register:                              2026-05-10, 2d
```

section Semana 2	
Endpoint generación de CV:	2026-05-12, 3d
Integración OpenAI en backend:	2026-05-15, 2d
Frontend formulario CV:	2026-05-17, 2d
Pruebas básicas CV:	2026-05-19, 1d
section Semana 3	
Endpoints estudio (resumen/flashcards):	2026-05-20, 3d
Integración IA de resúmenes:	2026-05-23, 2d
Frontend estudio (text input, mostrar resultados):	2026-05-25, 3d
section Semana 4	
Seguridad (JWT, filtros, CORS):	2026-05-27, 2d
Revisión UI / Diseño (Tailwind):	2026-05-29, 2d
Testing y debugging (unit tests, integración):	2026-05-31, 2d
Despliegue (Railway + Vercel) & checklist MVP:	2026-06-02, 2d

Checklist MVP (mínimo):

- Registro/login con JWT seguro.
- Generación básica de CV desde texto.
- Resumen de texto / flashcards.
- UI simple navegable (login, dashboard, formularios).
- Persistencia en DB (usuarios e historial).
- Despliegue funcional en plataforma cloud.
- Documentación mínima (README).

10. Seguridad, costos mínimos y límites de APIs

• Seguridad:

- Usar HTTPS en producción. Mantener secreta la llave de OpenAI (variable de entorno, no en frontend).
- Almacenar JWT en `HttpOnly` cookies para mitigar XSS, o usar `localStorage` con precauciones.
- Configurar Spring Security: rutas `/api/auth/**` públicas, otras protegidas ⁷. Implementar `JwtAuthenticationFilter` (extiende `OncePerRequestFilter`) para validar tokens ⁷.
- CORS: Permitir origen(es) frontend, incluir método `OPTIONS` en configuración de seguridad para evitar 401 preflight ⁸.

• Costes APIs:

- **OpenAI:** Los modelos modernos cobran por token. Ejemplo: GPT-5.4-mini cuesta **\$0.75 por 1M tokens de entrada** y **\$4.50 por 1M tokens de salida** ¹⁸. Un prompt/respuesta breve (200 tokens) costaría fracciones de centavo. Pueden usarse modelos más económicos o límites de uso (p.ej. 1-3 €/\$/mes gratis inicial).
- **HuggingFace:** Gratis hasta un nivel (usu básico) o pagos según uso. Útil para plantillas abiertas.

- **Límites:** Controlar `max_tokens` en solicitudes para evitar respuestas muy largas. Manejar cuotas (rate-limits) de API (p.ej. OpenAI tiene límites por minuto).
- **Costes fijos:** PostgreSQL en cloud (Railway tiene plan gratuito limitado). **Railway** ofrece PostgreSQL gratis hasta cierto tamaño de DB. **Vercel** y **Railway** tienen planes free para hobby projects.

11. Estrategias para primeros usuarios sin marketing pagado

- **Foros y comunidades:** Publicar en subreddits / Discord de desarrolladores y estudiantes (ej. r/learnprogramming, r/microSaaS, comunidades universitarias). Explicar beneficios (gestión de estudios/empleo).
- **GitHub / Product Hunt:** Si es open-source, invitar a probar y colaborar. Lanzar en Product Hunt con descripción breve y demo.
- **Redes sociales técnicas:** Compartir post en Twitter/X, LinkedIn enfocando cómo la IA ayuda a crear CV o estudiar. Mostrar capturas de pantalla.
- **Colaboraciones educativas:** Contactar escuelas o grupos de estudio; ofrecer su herramienta como caso de uso en proyectos o cursos.
- **Blog o tutorial:** Escribir un artículo "Cómo construir un asistente de estudio con IA" enlazándolo al repo, usando SEO (en español) para atraer tráfico orgánico.

12. Problemas comunes y cómo debuguearlos

- **CORS / 401 en Spring:** Si ves *"Response for preflight has invalid HTTP status code 401"*, revisar configuración CORS. Debes permitir rutas OPTIONS sin auth o usar `@CrossOrigin`. Baeldung describe solución para CORS en Spring Security ⁸.
- **JWT no válido:** Asegurar que el token se firma/verifica con la misma clave. Verificar que la fecha de expiración no sea muy corta. Logs de filtro muestran por qué falla (expirado, firma, etc).
- **Errores 500 en llamadas a OpenAI:** Pueden ser por clave API inválida o límite excedido. Revisar excepciones (`RestClientException`). Loggear respuesta HTTP completa ayuda.
- **Front-End: "Token" undefined:** Si guardas JWT en localStorage, refrescar página limpia el estado. Mejor usar estado global (Context) o cookie para persistir. Revisar consola si Axios no envía header.
- **Problemas de datos en JPA:** Errores de mapeo (por ejemplo, `id` duplicado) suelen ocurrir si no se configuran correctamente `@GeneratedValue`. Revisar mensajes de excepción SQL en logs.
- **Deploy fallido:** Chequear logs en Railway/Vercel. Común: no construir front (asegurar comandos `npm run build`) o no exponer puerto correcto en Spring (Railway detecta Dockerfile o puerto 8080). Verificar `Procfile` o configuración de Docker si se usa.
- **Git y Repositorios:** Incluye `.gitignore` para no subir credenciales. Si usas Railway one-click, "eject" el template para tu propio repo ¹⁹.

Consejo general: Buscar errores en StackOverflow o documentación oficial. Por ejemplo, la guía de Spring Boot sobre JPA ⁶ puede aclarar mapeos de entidades, y la documentación de React y Vite en español ¹⁰ ⁹ ayuda con problemas de configuración del frontend. Siempre dividir la aplicación (frontend/backend) y probar cada parte con herramientas de desarrollo y logs detallados.

Referencias: Documentación oficial y guías técnicas citadas en cada sección (Spring Boot ⁵, React ⁹ ¹⁰, Tailwind ¹¹, Axios ¹², OpenAI ¹³, HuggingFace ¹⁴, PostgreSQL ¹⁵, Railway ¹⁶, Vercel ¹⁷, estudios de caso ¹ ⁴ ³).

¹ ² ⁴ OpenAI enables student success for edtech giant | Cognizant

<https://www.cognizant.com/us/en/case-studies/edtech-openai-student-success>

³ New tools for understanding AI and learning outcomes | OpenAI

<https://openai.com/index/understanding-ai-and-learning-outcomes/>

⁵ Getting Started | Building an Application with Spring Boot

<https://spring.io/guides/gs/spring-boot/>

⁶ Getting Started | Accessing Data with JPA

<https://spring.io/guides/gs/accessing-data-jpa/>

⁷ Autenticación en Spring Security con JWT - Guía Completa | Jose López

<https://josealopez.dev/blog/authentication-with-spring-security-and-jwt>

⁸ Fixing 401s with CORS Preflights and Spring Security | Baeldung

<https://www.baeldung.com/spring-security-cors-preflight>

⁹ Inicio rápido – React

<https://es.react.dev/learn>

¹⁰ Crear una aplicación React desde cero – React

<https://es.react.dev/learn/build-a-react-app-from-scratch>

¹¹ Tailwind CSS (Spanish) - DEV Community

<https://dev.to/alfredocu/tailwind-css-spanish-4lk4>

¹² Cómo usar Axios con React | DigitalOcean

<https://www.digitalocean.com/community/tutorials/react-axios-react-es>

¹³ Using OpenAI ChatGPT APIs in Spring Boot | Baeldung

<https://www.baeldung.com/spring-boot-chatgpt-api-openai>

¹⁴ Access the Inference API · Hugging Face

https://huggingface.co/docs/huggingface_hub/v0.13.2/guides/inference

¹⁵ PostgreSQL: The world's most advanced open source database

<https://www.postgresql.org/>

¹⁶ ¹⁹ Deploy a Spring Boot App | Railway Guides

<https://docs.railway.com/guides/spring-boot>

¹⁷ React | Vercel

<https://vercel.com/solutions/react>

¹⁸ OpenAI API Pricing | OpenAI

<https://openai.com/api/pricing/>